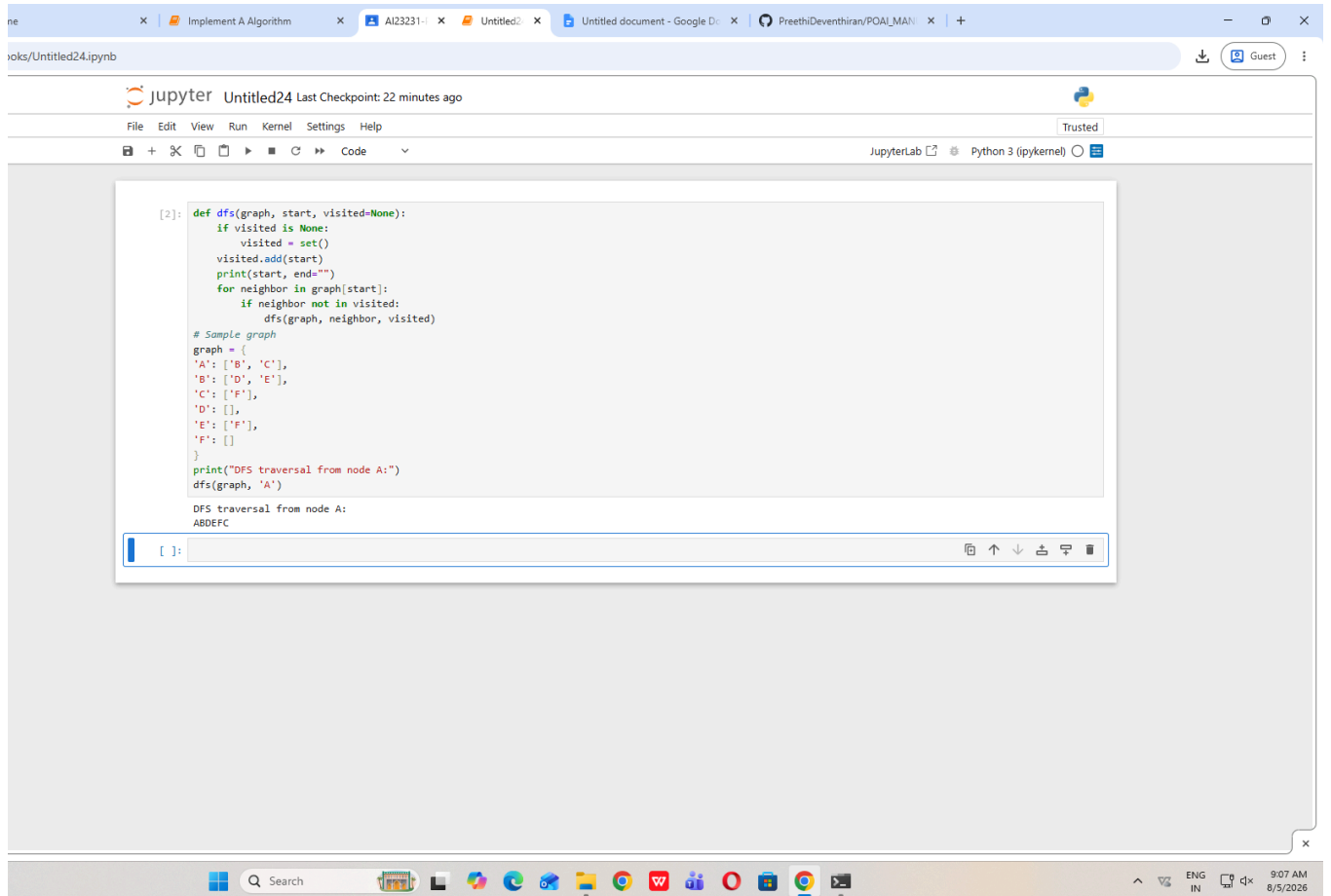


EX 02

Solve a Graph using Depth First Search (DFS)

241001176



The screenshot shows a JupyterLab environment with a code editor and a console output. The code implements a Depth First Search (DFS) algorithm on a graph. The graph is defined as follows:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
```

The DFS function is defined as:

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

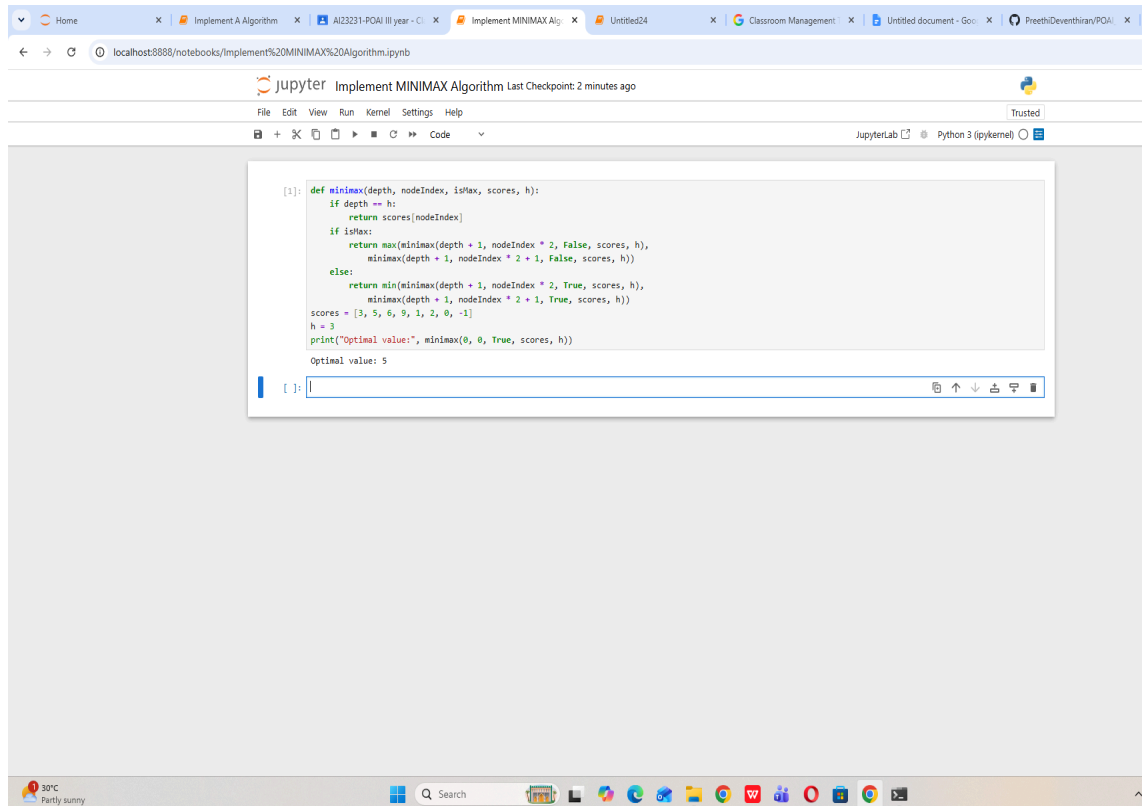
The code is executed, and the output in the console is:

```
DFS traversal from node A:
ABDEFC
```

EX 03

Implement MINIMAX Algorithm

241001176



The screenshot displays a JupyterLab environment with a single code cell. The code implements a Minimax algorithm for a game tree. The function 'minimax' is defined with parameters: depth, nodeIndex, isMax, scores, and h. It uses a list 'scores' with values [3, 5, 6, 9, 1, 2, 0, -1] and a height 'h' of 3. The function prints 'Optimal value: 5'.

```
[1]: def minimax(depth, nodeIndex, isMax, scores, h):  
    if depth == h:  
        return scores[nodeIndex]  
    if isMax:  
        return max(minimax(depth + 1, nodeIndex * 2, False, scores, h),  
                    minimax(depth + 1, nodeIndex * 2 + 1, False, scores, h))  
    else:  
        return min(minimax(depth + 1, nodeIndex * 2, True, scores, h),  
                    minimax(depth + 1, nodeIndex * 2 + 1, True, scores, h))  
    scores = [3, 5, 6, 9, 1, 2, 0, -1]  
    h = 3  
    print("Optimal value:", minimax(0, 0, True, scores, h))  
    Optimal value: 5
```